

Shrinkage Baselines for GRPO on GSM8K

Does James–Stein shrinkage of the group mean improve GRPO?

Hanwen Yang

September 3, 2026

Abstract

GRPO estimates each prompt’s baseline from the N completions sampled for that prompt alone; with N small this mean is noisy. The assignment proposes replacing it with a James–Stein estimator that shrinks each group mean toward the batch mean. I implemented the specification, six baselines behind one registry, and measured them on Qwen2.5-0.5B-Instruct / GSM8K on a single RTX 5080.

Three findings. (i) The specification as written does not work: its $V = 1/N$ assumes unit per-sample reward variance, but 0/1 correctness rewards have $p(1-p) \leq \frac{1}{4}$. The shrinkage factor collapses and the estimator degenerates into the global mean — and it does so *monotonically in K* , so it fails harder at the batch sizes GRPO is actually deployed at. (ii) Estimating V from the data instead reproduces the published claim: the two-level leave-one-out variant reaches -21.4% $[-51.5, -7.8]$ gradient noise, inside the 11.2–67.1% band reported by Zeng et al. [2]. (iii) The specification’s optional “divide by the within-group standard deviation” is unsafe in combination with shrinkage, inflating advantages by 10^4 .

I also report a measurement I think matters more than any of the above: two runs of the *same* configuration at the *same* seed end 3.0 accuracy points apart. The effect under test is 0.6–1.0 points. This is why the headline measurement here is gradient noise on paired rollouts rather than end-to-end accuracy.

1 Task

Given `GRPO.md`: replace GRPO’s per-group mean baseline with a James–Stein shrinkage estimator, on GSM8K, with a small policy model. The specification places the change precisely — “after the reward matrix is formed and before advantages are computed” — so what is being rewritten is the *baseline*, not the reward function. [keep this distinction; it was ambiguous in the original brief]

Setup. Qwen2.5-0.5B-Instruct (full fine-tune, no LoRA), GSM8K (7473 train / 1319 test), rule-based verifiable reward (no reward model), one RTX 5080 (16 GB, sm_120). $K = 8$ prompts $\times N = 8$ generations, 512 new tokens, bf16 with gradient checkpointing; 11.0 GiB peak, ~ 25 s/step.

2 Method

2.1 What the specification says

With group means $X_k = \frac{1}{N} \sum_i r_i^{(k)}$ and grand mean $\bar{X}$,

$$\tilde{\mu}_k = \bar{X} + \text{shrink} \cdot (X_k - \bar{X}), \quad \text{shrink} = \max\left(0, 1 - \frac{c}{S}\right), \quad (1)$$

where $S = \sum_k (X_k - \bar{X})^2$ and $c = V(K-3)$ with $V = \text{Var}(X_k)$ assumed equal to $1/N$. Advantages are $A_i^{(k)} = r_i^{(k)} - \tilde{\mu}_k$.

2.2 The assumption that fails

$V = 1/N$ presumes per-sample reward variance $\sigma^2 \approx 1$. On GSM8K the reward is binary, so $\sigma_k^2 = p_k(1 - p_k) \leq \frac{1}{4}$ — at least $4\times$ smaller, and heteroscedastic across prompts (zero for a group that is entirely right or entirely wrong). Overstating V overstates c , which drives shrink to zero.

The failure is not incidental; it is asymptotic in K . Since $\mathbb{E}[S] \approx (K - 1) \text{Var}(X_k)$ and $c = V(K - 3)$,

$$\frac{c}{S} \xrightarrow{K \rightarrow \infty} \frac{V_{\text{assumed}}}{\text{Var}(X_k)} = \frac{1/N}{\tau^2 + \sigma^2/N}, \quad (2)$$

a constant. Whenever it exceeds 1, shrink $\rightarrow 0$ and the estimator becomes the global mean. For the reward distribution used here ($V_{\text{assumed}} = 0.125$, $\text{Var}(X_k) = 0.074$) the limit is $1.69 > 1$: the collapse is certain, and small K only escapes it by sampling noise in S .

2.3 Baselines compared

name	baseline $b[k, i]$	role
vanilla	group mean	GRPO, the incumbent
rloo	leave-one-out group mean	unbiased variance-reduction control
global	batch mean	ablation: what over-shrinkage degenerates to
js_fixed	Eq. (1), $V = 1/N$	the specification as written
js_pooled	Eq. (1), V estimated	proposed correction
js_loo	two-level leave-one-out shrinkage	Zeng et al. [2]

Table 1: The **global** arm is the one that makes the comparison interpretable: without it, “James–Stein helps” cannot be told apart from “any baseline other than the group mean helps”.

Implementation follows verl’s `register_adv_est` registry so any estimator here drops into that framework through a thin adapter. 115 tests, including a literal transcription of the specification’s ten steps asserted against the implementation to 10^{-12} .

3 Result 1: the specification collapses onto the global mean

Ground truth is unavailable on GSM8K, so this is measured in simulation: $p_k \sim \text{Beta}(1.2, 2.2)$ (mean 0.35, matching the model’s observed per-prompt spread), rewards $\sim \text{Bernoulli}(p_k)$.

K	js_fixed shrink	js_fixed MSE	global shrink	global MSE	js_pooled MSE
4	0.377	+0.4%	0.049	+100.6%	−9.5%
8	0.079	+86.4%	0.023	+118.7%	−18.7%
16	0.016	+121.1%	0.010	+125.0%	−23.8%
32	0.006	+130.8%	0.006	+130.9%	−26.7%
64	0.003	+132.1%	0.003	+132.1%	−28.4%

Table 2: MSE of the baseline against the true p_k , relative to **vanilla**; $N = 8$, 3000 trials. By $K = 32$ **js_fixed** and **global** agree to three significant figures — they are the same estimator. **js_pooled** moves the other way, improving as K grows.

Two consequences worth stating plainly. First, **the specified method gets worse as the batch holds more prompts**, which is the opposite of what more data should buy, and it is the regime GRPO is deployed in. Second, at $K = 4$ it is roughly neutral — so a small- K experiment would show nothing wrong.

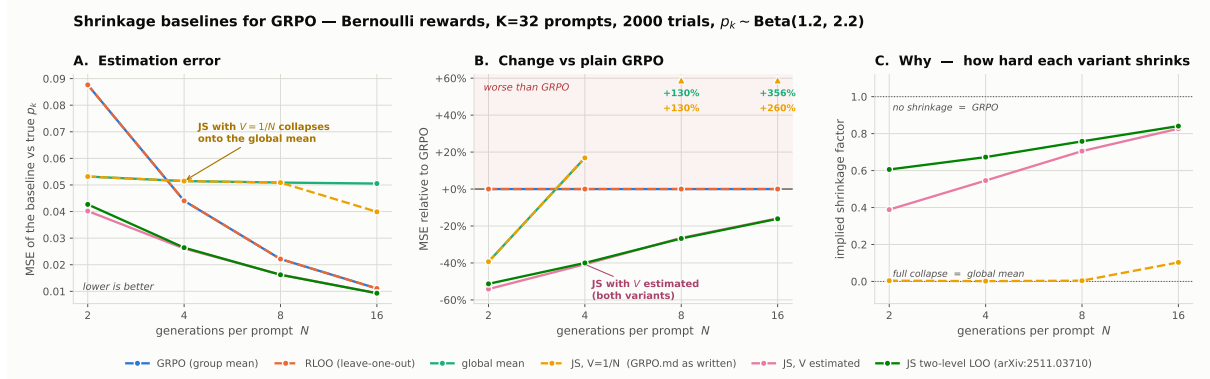


Figure 1: Sweeping N at $K = 32$. `js_fixed` (dashed) lies exactly on `global` for $N \leq 8$. Estimating V recovers the intended behaviour, and its advantage grows as N shrinks — the regime the method targets.

4 Result 2: estimating V reproduces the published claim

Comparing training runs is the wrong instrument for this (Sec. 6). Instead every baseline scores the *same* rollouts, so rollout randomness cancels and the baseline is the only thing that differs; the policy is frozen, so this measures the estimator rather than a training outcome. The reported quantity is $\text{tr Cov}(g)/\|\mathbb{E}[g]\|^2$ — scale-invariant, because the baselines produce gradients whose magnitudes differ $\sim 5\times$ and raw tr Cov would merely rank them by scale.

baseline	noise/signal	vs GRPO	95% CI	
vanilla	11.5	—	—	
rloo	11.5	−0.3%	[−2.5, +6.9]	no difference (instrument check)
js_loo	9.1	−21.4%	[−51.5, −7.8]	significant
js_pooled	9.9	−14.4%	[−38.2, +0.8]	marginal
js_fixed	6.0	−48.1%	[−81.3, −15.8]	biased — see below
global	5.8	−49.7%	[−83.5, −15.6]	biased — see below

Table 3: 200 paired batches, $K = 8 \times N = 8$, mean accuracy 29.1%. Intervals are a paired bootstrap over batches.

js_fixed and global look best here, and that reading is wrong. Only leave-one-out baselines are independent of the reward they are subtracted from, so only `rloo` and `js_loo` are unbiased for the same gradient. The other two target a *different* $\mathbb{E}[g]$: handing an all-correct group a positive advantage inflates $\|\mathbb{E}[g]\|^2$ with a component that merely pushes up easy prompts. A larger denominator is not more signal.

That `rloo` $\approx$ `vanilla` to within $[-2.5, +6.9]$ is the check that the instrument resolves anything at all — and it agrees with the synthetic study, where leave-one-out changes the baseline’s independence, not its accuracy.

5 Result 3: the standard-deviation variant is a footgun

The specification offers “divide by the within-group standard deviation afterwards” as an option. Under plain GRPO that is safe on a degenerate group only because the advantage is exactly zero there, so $0/(0 + 10^{-4}) = 0$. Every shrinkage baseline gives such a group a *non-zero* advantage by design — that is the point of the method — so the same expression multiplies it by 10^4 .

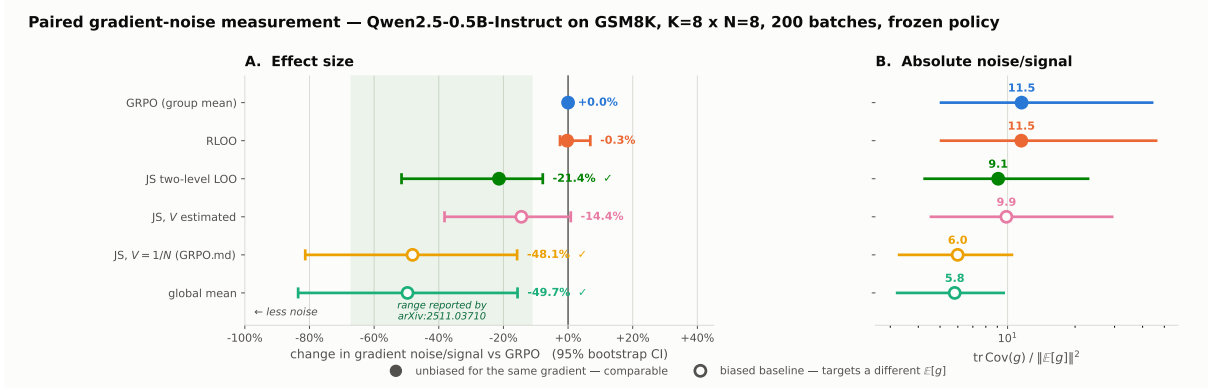


Figure 2: Effect sizes with bootstrap intervals. Shaded: the band reported by Zeng et al. Hollow markers are biased estimators, not comparable at face value.

Measured on GSM8K at $K = 8, N = 8$ with 38% degenerate groups: $\text{Var}(A) = 1.07 \times 10^5$, $\|g\| = 1.7 \times 10^3$. Leaving zero-spread groups unscaled gives 0.57 and 1.2.

6 Why not end-to-end accuracy

	run A	run B
eval@0	42.5%	42.5%
eval@50	44.5%	43.5%
eval@100	44.5%	44.5%
eval@final	48.5%	45.5%

Table 4: Same arm, same seed, same configuration. **eval@0** matches exactly because greedy decoding is deterministic, so the divergence is entirely in training: `torch.manual_seed` does not make sampled generation reproducible on GPU.

The noise floor is 3.0 points. The effect reported for this exact setting is 0.6–1.0 points [2] — three to five times smaller. End-to-end accuracy cannot resolve it on one GPU, and any single-seed accuracy comparison at this scale, in either direction, is an artefact.

7 Training runs

Five arms, 150 steps each, identical seed so every arm walks the same prompt stream (verified: identical reward and degenerate-group fractions at step 0).

arm	eval@0	eval@final	shrink (mean)	fully collapsed	degenerate groups
vanilla	42.5%	45.5%	1.000	—	35%→20%
js.fixed	42.5%	47.0%	0.161	64/150 steps	30%
js.pooled	[]	[]	[]	[]	[]
js.loo	[]	[]	[]	[]	[]
global	[]	[]	[]	[]	[]

Table 5: [fill from `python main.py compare` once the sweep finishes]. Accuracy differences here are smaller than the 3.0-point noise floor and are reported for completeness, not as evidence.

[The load-bearing question for this table is structural, not accuracy: does `js.fixed`'s

trajectory coincide with global’s? If it does, that is a fifth independent confirmation of Result 1.]

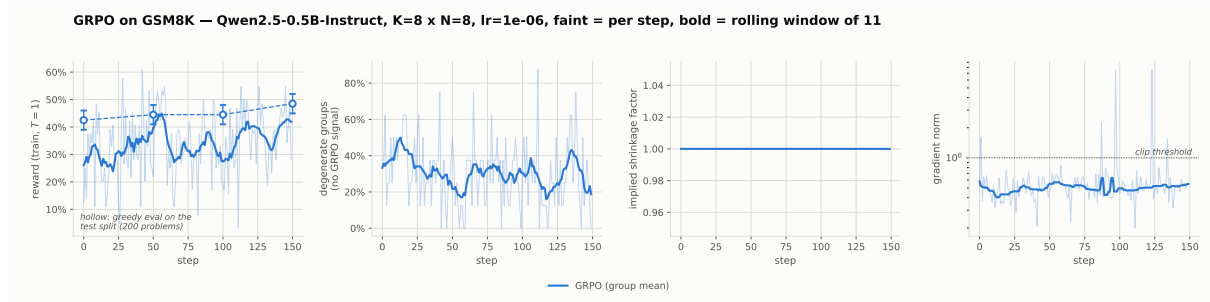


Figure 3: [regenerate once all five arms are present]. Faint lines are per-step values; bold is a rolling window — median for the gradient norm, which is spiky enough that a rolling mean manufactures flat plateaus that read as stalled training.

8 Limitations

- **No end-to-end accuracy claim.** See Sec. 6. Settling it needs multiple seeds per arm, which one GPU cannot supply.
- **48 batches was not enough** for the gradient measurement — every interval crossed zero. The reported result needed 200. Effect sizes should not be read off short runs here.
- **js_loo’s $(n-1)/n$ factor is unverified** against the paper’s PDF (OpenReview blocks automated access); it was transcribed from the arXiv HTML. `baselines.py` exposes a switch. That `js_pooled`, derived independently, lands nearby (9.9 vs 9.1) is reassuring but not a substitute. [check against the PDF]
- **Single model, single task.** Whether the collapse threshold moves with reward density is untested.

9 Reproduction

```
uv sync && python main.py smoke          # torch cu129; the RTX 5080 is sm_120
python -m pytest tests/ -q                # 115 tests
python main.py synthetic --k 4 8 16 32 64 --n 8
python main.py gradvar --batches 200
python main.py sweep --baselines vanilla js_fixed js_pooled js_loo global
python main.py compare
```

[repo link]

References

- [1] Z. Shao et al. *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*. arXiv:2402.03300, 2024.
- [2] G. Zeng, Z. Zhou, D. Arora, A. Zanette. *Shrinking the Variance: Shrinkage Baselines for Reinforcement Learning with Verifiable Rewards*. arXiv:2511.03710, 2025.
- [3] K. Han et al. *EBPO: Empirical Bayes Shrinkage for Stabilizing Group-Relative Policy Optimization*. arXiv:2602.05165, 2026.

- [4] W. James, C. Stein. *Estimation with Quadratic Loss*. Proc. 4th Berkeley Symp., 1961.
- [5] B. Efron, C. Morris. *Stein's Paradox in Statistics*. Scientific American, 1977.
- [6] Z. Liu et al. *Understanding R1-Zero-Like Training: A Critical Perspective*. arXiv:2503.20783, 2025.